

UI design of my harness

Files and folder

Main chat interface

progress

The screenshot displays the LordBlack harness v0.8.4 interface, which is divided into three main sections: Files and folder, Main chat interface, and progress.

Files and folder: The left sidebar shows the workspace structure. The file tree includes `/workspace`, `src/` (containing `__init__.py`, `main.py`, `config.py`, and `utils.py`), and `tests/` (containing `test_utils.py`). The `utils.py` file is selected. Below the file tree, the **MEMORY RETRIEVAL** section lists project details: Project Stack (Python, FastAPI, Pydantic), Environment (local PostgreSQL on port 5432), User preferences (typing, async, AWS), and Rule (Ensure schemas are validated).

Main chat interface: The central panel shows the **CHAT & WORKSPACE** section. It displays the current models (Claude 3.5 Sonnet (External) + Llama 3.1 8B (Local)) and the chat history. The chat content includes a prompt to refactor `utils.py` to load configuration from `schema.json` using `pydantic`, followed by the implementation code for `AppConfig` and `load_config` functions. The code is as follows:

```
import json
from pydantic import BaseModel

class AppConfig(BaseModel):
    host: str = "127.0.0.1"
    port: int = 8000
    debug: bool = False

def load_config(path: str) -> AppConfig:
    with open(path, "r") as f:
        data = json.load(f)
    return AppConfig(**data)
```

Below the code, there are buttons for **Apply to File** and **View Diff**. The **Live Agent Run Trace** section shows the progress of the agent's execution, including `Locating schema.json...` (Done) and `Modified src/utils.py...` (Completed).

progress: The right sidebar shows the **AI ACTIVITY** section, which displays the progress of the AI agent's execution. The activity is divided into several steps: **START: Input parsed** (Success), **LOCAL QWEN MODEL** (Analyzing routing & intent, Done, 74ms), **MEMORY SEARCH** (Queried PGVector 'archives_2024', Found 3 chunks, 120ms), **EXTERNAL API MODEL** (Routing to gpt-4o-mini for validation, Connected, 840ms), **RESPONSE EXTRACTION** (Parsing pattern blocks, Extracting, 65ms), and **FINAL ANSWER GENERATION** (Synthesizing visual report, Streaming, 0.4s). The bottom of the sidebar shows a log of system messages, including `INFO [system] Initiated agent pipeline`, `DEBUG [local_qwen] Router routing to vector search`, `MEMORY [pgvector] Found high-similarity matches`, `API [gpt-4o-mini] Dispatching validation request`, and `PARSER [extractor] Identified pattern anomaly`.



Inside settings, there will be the configuration interface like this which will allow the user to connect harness with lm studio and also do all other configurations like connecting APIs, connecting a local model with cloud models etc.

